SYSTEM VULNERABILITY MANAGEMENT (CYS330)

Understanding logic Gates/Circuits

A Presentation by Cybersecurity Tutorials Kapasa Makasa University 3rd Year Cybersecurity Students

Prepared for Mr. Mutunda

Prepared by Evans B Devancie for cyber security tutorials @KMU

LOGIC GATES & TRUTH TABLES

- These are fundamental building blocks of digital circuits, performing basic logical operations on binary inputs (0s or 1s) to produce a binary output.
- Forming the basics for complex computer systems and electronic devices

Types

- **AND GATE**
- Performs logical multiplication of binary input.
  - ❑ The output state of the AND GATE will be high (1)
  - ❑ If both output are high(1) “1 x 1”
  - ❑ else the output will be low (0) If any of the input is low(0)

The value of X will be true when both the input will be True

CONT.....

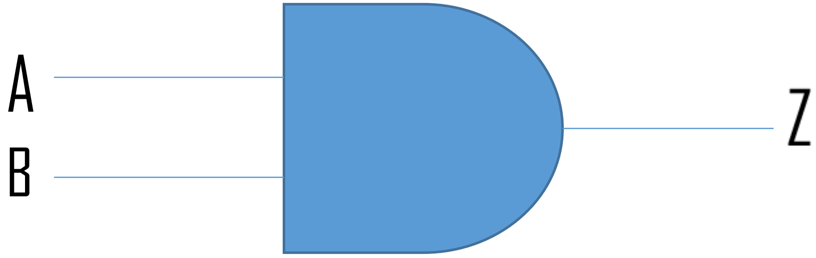
- It is an electronic circuit having one or more than one input and only one output. The relationship between the input and the output is based on a certain logic. Based on this, logic gates are named as AND gate, OR gate, NOT gate etc. (Enderton, 2001).
- A truth table is a mathematical table used in logic—specifically in connection with Boolean algebra, Boolean functions, and propositional calculus—which sets out the functional values of logical expressions on each of their functional arguments, that is, for each combination of values taken by their logical variables (Enderton, 2001).

AND GATE Cont'd

- Logical Notations
  - Z = A AND B
- Boolean algebra
  - Z = A.B
- A (input 1)   B(input 2)   Z=(A.B)

0	**1**	**0**
1	0	0
1	1	1
0	0	0

symbol (x)



OR GATE

- Is the most widely used digital logic circuit
- The output state of OR GATE will be high i.e. (1) if any of output state is high i.e. 1 else output state will be low i.e. 0 X
- Boolean expression
- X = A + B

- The value X will be high (true) when one of the inputs is set to high (true)
- Input (A) input (B) output (X)

Input (A)	input (B)	output (X)
1	0	1
0	1	1
1	1	1
0	0	0

Logical Notations

X = A OR B

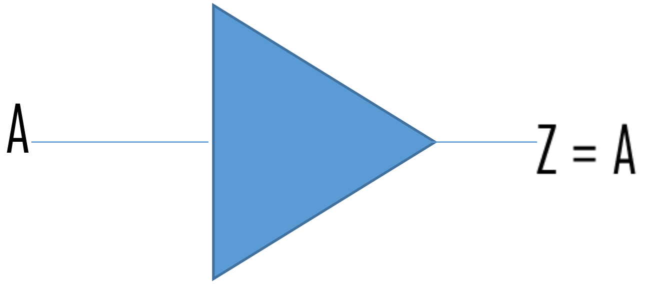
Boolean

X = A + B

NOT

- **NOT GATE**
- In digital electronics the NOT GATE is one of the basic logic GATE having only a single input and a single output, Its also known as an inverter or inverting buffer.
- When the input single is **low** the output single is **high** and vice-versa
- The Boolean expression of NOT GATE is as follows
  - Y = A′ or
  - Y = A′
- The value of Y will be high when A will be low
- A (input)   Z = A′ (output)

0	1
1	0



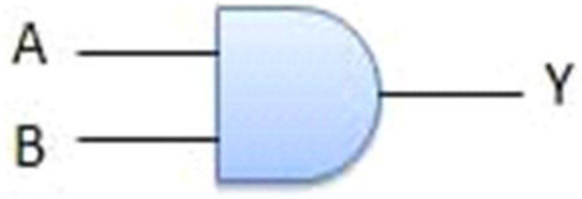
Not Gate takes only one output

the output of a not gate is a complement or inverse of the input applied to it.

Summary

NAME LOGIC DIAGRAM TRUTH TABLE

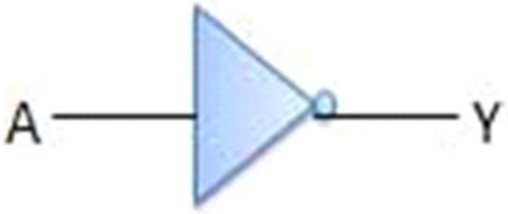
AND



Inputs		Output
A	B	AB
0	0	0
0	1	0
1	0	0
1	1	1

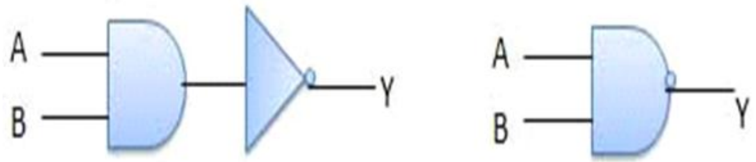
OR

NOT



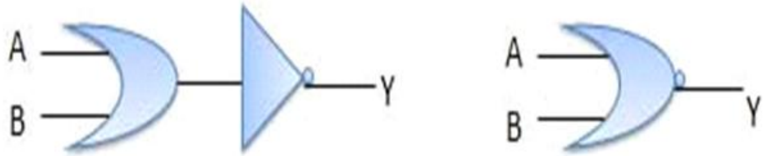
Inputs	Output
A	B
0	1
1	0

NAND



Inputs		Output
A	B	$\overline{AB}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR



Inputs		Output
A	B	$\overline{A+B}$
0	0	1
0	1	0
1	0	0
1	1	0

XOR

Inputs		Output
A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

XNOR

Inputs		Output
A	B	$A \ominus B$
0	0	1
0	1	0
1	0	0
1	1	1

Binary Number system
understanding overflow with calculations

- A computer uses digits but there is time that a computer switches to octal, decimal and hexadecimal number system.
- Octal and hexadecimal will be skipped you can learn on your free time.
- Will look at Binary Number system
- Addition of Binary Numbers – we have two elements 0s and 1s
- Example 1

```
  1 1 0 1
+   1 1 1
---------
1 0 1 0 0
```

Example 2

```
 1 1 1 1
 0 1 1 1
--------
1 0 1 1 0
```

Prepared by Evans B Devancie for cyber security tutorials @KMU

HOW TO CONVERT A DECIMAL NUMBER INTO BINARY (POWERS OF 2)

- To achieve this will utilize the powers of 2.
- 2 ^ 0 = 1
- 2 ^ 1 = 2
- 2 ^ 2 = 4
- 2 ^ 3 = 8
- 2 ^ 4 = 16
- 2 ^ 5 = 32
- 2 ^ 6 = 64
- 2 ^ 7 = 128

```
>>> bin(75)
'0b1001011'
>>>
```

- For example, let's convert 75 into binary
- 128 64 32 16 8 4 2 1

0 | 1 0 0 1 0 1 1 = 75

Sign (+) Magnitude

HOW TO CONVERT A DECIMAL NUMBER INTO BINARY (POWERS OF 2) (CONTINUES…)

- Let's take another example of 339

- 1 = True
- 0 = False

```
>>> int (True)
1
>>> int (False)
0
>>>
```

```
>>> bin(339)
'0b101010011'
>>>
```

HOW TO CONVERT A DECIMAL NUMBER INTO BINARY SUCCESSIVE DIVISION

Now, let's use successive division to find the binary of 75.

75 / 2 = 37 R 1

37 / 2 = 18 R 1

18 / 2 = 9 R 0

9 / 2 = 4 R 1

4 / 2 = 2 R 0

2 / 2 = 1 R 0

1 / 2 = 0 R 1

- We now get the binary from the bottom to the top, making it:
- 1001011

```
>>> bin(75)
'0b1001011'
>>> 0b1001011
75
>>>
```

HOW TO ADD BINARY NUMBERS

- Here are some basics of addition:
- 0 + 0 = 0
- 0 + 1 = 1
- 1 + 0 = 1
- 1 + 1 = 10

    1
+ 1
___
1 0 , this is the same as 1 + 1 = 2
  - 2 / 2 = 1 R 0

```
>>> bin(1+1)
'0b10'
>>>
```

HOW TO ADD BINARY NUMBERS (CONTINUES…)

  1

  1

\+ 1

_____

1   1 This is the same as 1+1+1 = 3

- 3 / 2 = 1 R 1

```
>>> bin(1+1+1)
'0b11'
>>>
```

- NOTE: On the bottom we are putting the reminder (R) and carrying the number of times 2 enters into the target number

HOW TO ADD BINARY NUMBERS (CONTINUES…)

- Let's take an example below:

  1010 = 10

  \+ 1001 = 9

  ______ ___

  10011 = 19

16	8	4	2	1	Decimal
	1	0	1	0	10
	1	0	0	1	9
1	0	0	1	1	19

```
>>> bin(10)
'0b1010'
>>> bin(9)
'0b1001'
>>> bin(19)
'0b10011'
>>>
```

HOW TO SUBTRACT BINARY NUMBERS

Here are some basics of subtraction:

1 - 0 = 1

1 - 1 = 0

```
>>> bin(8-1)
'0b111'
>>>
```

1 0 → 0 2

\- 1 → 1

	1	1	
0	2	2	2
1	0	0	0
-			1
0	1	1	1

→

8	4	2	1
8			
-			1
			7

1 In base 2, will get a 2 from the nearest value

1'S & 2'S COMPLEMENT

1'S COMPLEMENT

- Obtain 1's complement of 1010.
- NOTE: In ones complement we flip the numbers, 1 becomes 0 and 0 becomes 1.
- Therefore 1's complement of 1010 is 0101

2'S COMPLEMENT

- Formula:

2's complement = 1's complement + 1

QUESTION: Obtain 1's complement of 10111010.

1s complement = 01000101

2s complement = 01000110

SIGNED MAGNITUDE

CONVERTING A BINARY

- Let's take an example, let's express (-6)

using the signed magnitude.

1. What is (+6) in binary?

8 4 2 1

0 1 1 0 = +6

Magnitude

Sign (+)

NEGATIVE BINARY REPRESENTATION

2. The Sign (+) will change from 0 to 1 Sign (-).

8 4 2 1

Sign (-) ← 1 1 1 0

NOTE: 1 represent negative and 0 positive

2'S COMPLEMENT

NEGATIVE NUMBERS

- Represent -12 in binary using the 2's complement.

16 8 4 2 1

0 1 1 0 0 = 12

1 0 0 1 1 = 1's complement

2'S COMPLEMENT (-12)

1 0 0 1 1

\+ 1

____

1 0 1 0 0 = -16 + 4 = -12

Sign (-) Magnitude

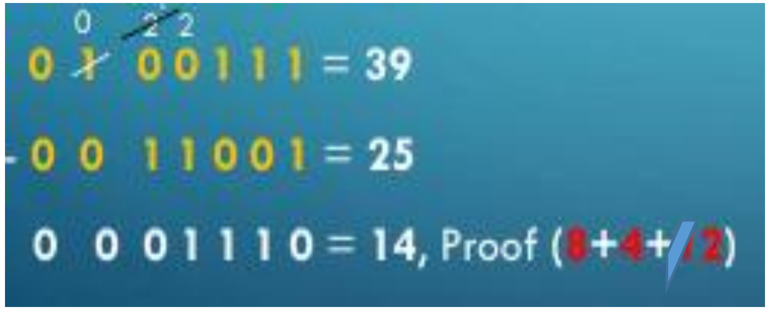
2'S COMPLEMENT SUBTRACTION

BINARY CONVERSATION

- Let's subtract 39 – (25).
- Using normal math operation 39 – 25 = 14.

1. Convert 39 and 25 into binary.

64 32 16 8 4 2 1



1'S & 2'S COMPLEMENT

2. Find the 1's complement 25.

1 1 0 0 1 1 0 = 25 1's complement

3. Find the 2's complement of 25.

1 1 0 0 1 1 1 = 25 2's complement

NOTE: We are targeting **25**, because in 2's complement subtraction we **change the sign** of the number after the minus (-) sign, so **25** becomes **-25**.

2'S COMPLEMENT SUBTRACTION (CONTINUES...)

2'S COMPLEMENT SUBTRACTION

4. Finally, do the 2's complement subtraction

of 39 and 25.

0 1 0 0 1 1 1

\+ 1 1 0 0 1 1 1

(1) 0 0 0 1 1 1 0

NOTE: We have a carry 1 at the end,

but we are just ignoring it, meaning an Overflow as occurred.

VERIFICATION

- Is 0001110 14?

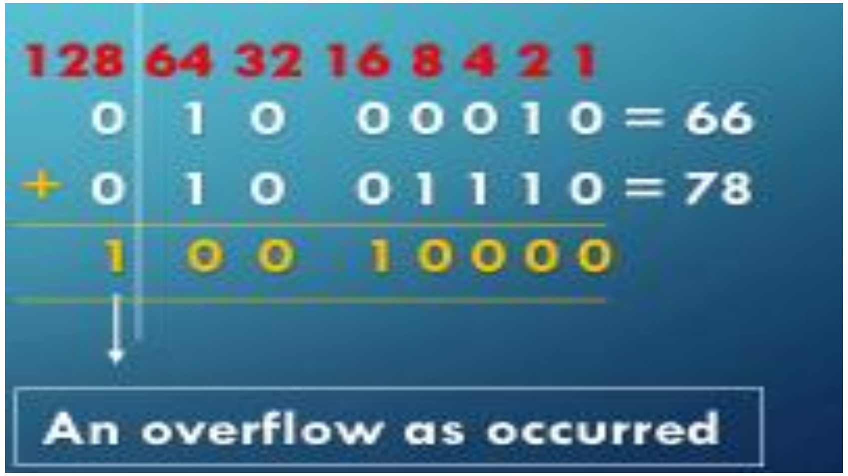
128 64 32 16 8 4 2 1

0 0 0 0 1 1 1 0

- 8 + 4 + 2 = 14, NOTE: In 2's complement subtraction we add two binaries, think of it as 39 – 25 is the same as 39 + (-25)

2'S COMPLEMENT SUBTRACTION (Example 2)

1. 66 – (-78) = 144
2. This is the same as 66 + (78)



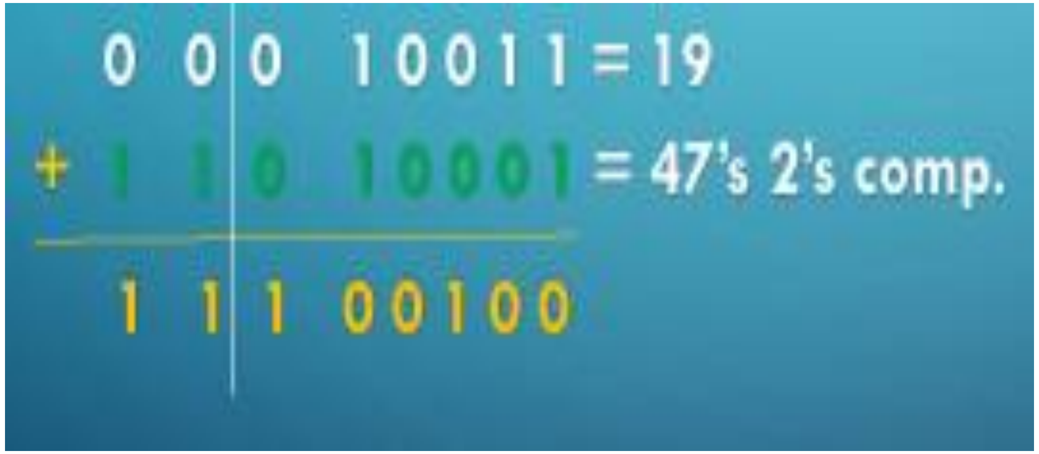
Intentionally left blank

2'S COMPLEMENT Addition (example 1)

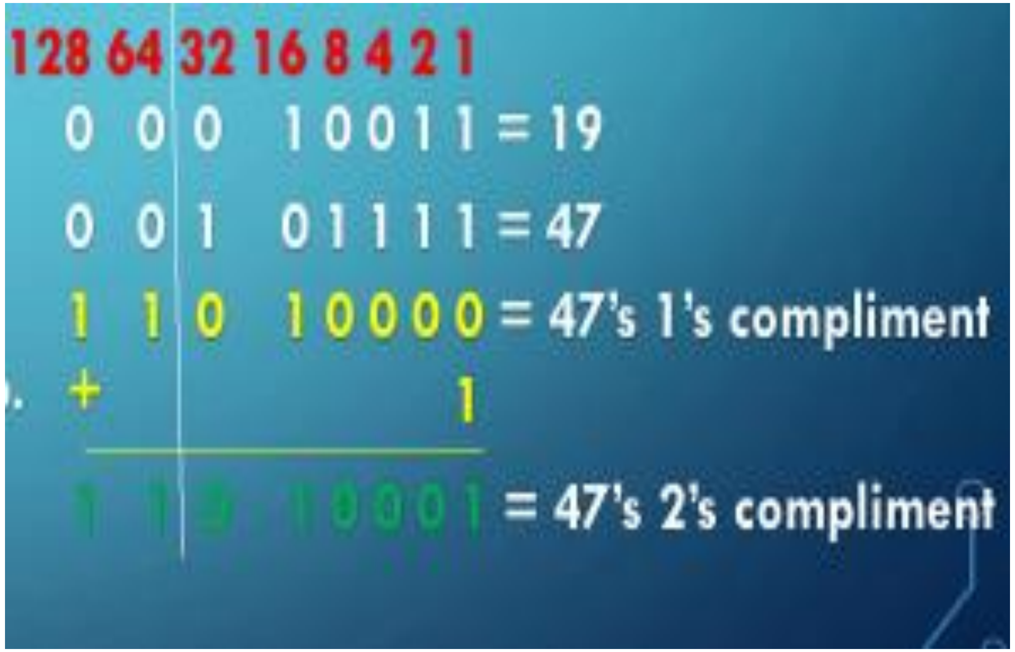
1. -47 and 19

- 47 + 19 or 19 – 47 = -28

19 + (-47)



$

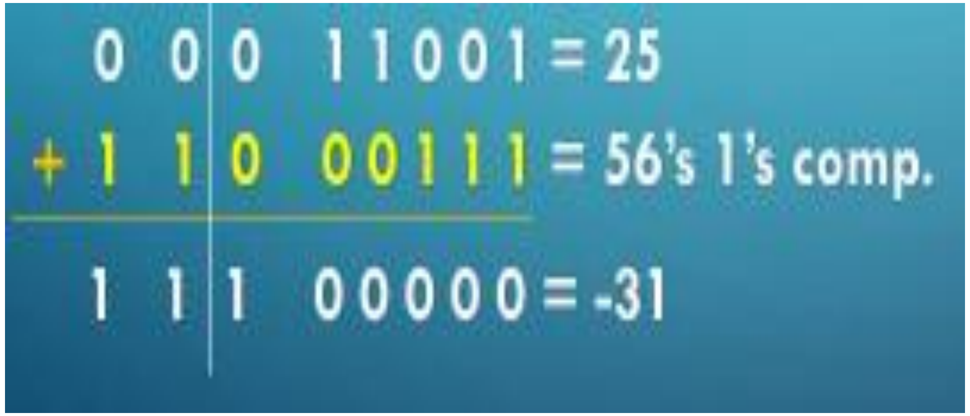


2'S COMPLEMENT Addition (example 2)

1. 25 and -56

$

25 + (-56) = -31



QUESTIONS ?